Redes de Computadores II

Curso 22/23 :: Prueba 1 (extraordinaria)

Escuela Superior de Informática

2026/03/18 12:52:41

Este examen suma un total de 20 puntos. Cada 3 preguntas de test incorrectas restan 1 punto. Sólo una opción es correcta a menos que se indique algo distinto. No está permitido el uso de calculadora. La duración del examen es de 40 minutos.
Siga las instrucciones de la hoja de respuestas.
Marque sus respuestas sólo cuando esté completamente seguro. El escáner no admite correcciones ni tachones de ningún tipo, las anulará automáticamente. Debe entregar únicamente la hoja de respuestas.

1 [1p] ¿Cuál es la función principal del protocolo ARP en TCP/IP?

- ☐ **a)** Encapsular conexiones TCP/IP sobre medio físico aéreo con el protocolo 802.11 (WiFi).
- ☐ **b)** Traducir nombres de dominio en direcciones IP.
- ☐ **c)** Establecer una asociación entre la dirección MAC y la dirección IP de cada equipo.
- ☐ **d)** Establecer conexiones TCP entre hosts y garantizar la entrega de paquetes.

2 [1p] ¿Cuál es el propósito del protocolo ICMP en TCP/IP?

- ☐ **a)** Asegurar que un socket, una vez abierto, no pierde la conectividad.
- ☐ **b)** Facilitar la resolución de direcciones IP.
- ☐ **c)** Establecer conexiones TCP entre hosts.
- ☐ **d)** Realizar pruebas de conectividad y reportar errores en la red.

3 [1p] De las siguientes primitivas Python para crear un servidor UDP:

```
bind()
socket()
sendto()
recvfrom()
```

- ☐ **a)** Falta la función close() que cierra el socket una vez deja de usarse, lo demás es correcto.
- ☐ **b)** Puesto que es un servidor el único error es que primero debería escuchar con `recvfrom()` para posteriormente enviar con `sendto()`.
- ☐ **c)** Primero habría que definir el `socket()`, luego asignarlo `bind()` y adicionalmente intercambiar el orden de `recvfrom()` y `sendto()`.
- ☐ **d)** Las funciones descritas corresponden a un servidor TCP, no UDP.

4 [1p] Los sockets UDP transmiten datagramas, los TCP un flujo contínuo de información ¿y los RAW?

- ☐ **a)** Son una variante de TCP, pero con más flexibilidad a la hora de programarlos.
- ☐ **b)** Sólo existen en sistemas Linux.
- ☐ **c)** Es un socket genérico para programar en Python código reutilizable. En la llamada a funciones con este tipo de socket hay que especificar si DGRAM o STREAM.
- ☐ **d)** Es un socket mucho más versatil y complejo de programar, pero que al entrar directamente al nivel de red, permite funciones adicionales.

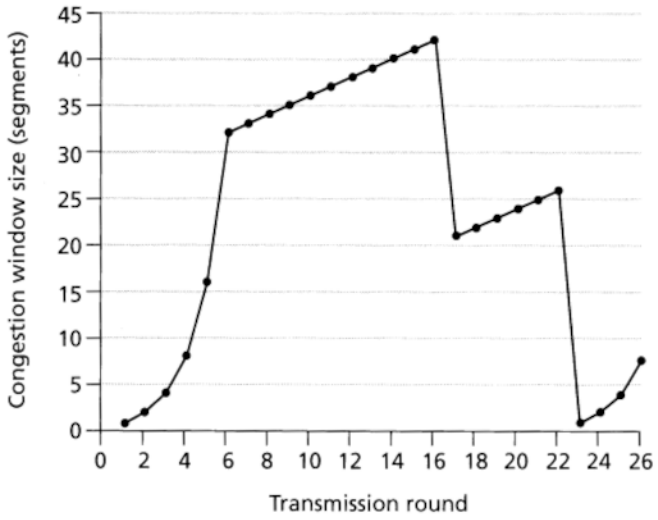
5 [1p] Los Sockets pueden tratarse como ficheros, de manera que pueden usarse métodos de lectura y escritura como tales, simplificando mucho su manejo:

- ☐ **a)** Sí pero sólo en C y C++.
- ☐ **b)** Sí pero en C, C++ y Java.
- ☐ **c)** Sí en C, C++, Java y Python.
- ☐ **d)** No, es un error muy frecuente: los ficheros permiten acceso aleatorio y los sockets sólo secuencial.

6 [1p] Un socket de la familia INET, **siempre** está definido por una dirección IP y un puerto.

- ☐ **a)** FALSO: puede estar asociado a más de un puerto.
- ☐ **b)** FALSO: podemos no asociarlo. Hay sockets en los que no se especifica puerto y funcionan igual.
- ☐ **c)** VERDADERO: la definición de socket, *conector* es precísamente una IP específica y un puerto o servicio al que acceder.
- ☐ **d)** VERDADERO: y adicionalmente, la IP debe ser pública y el puerto no estar cerrado.

A [7p] Considere el siguiente gráfico que representa la ventana de congestión de una conexión TCP. Responda a las siguientes preguntas:



> **7** (1p) Identifica los intervalos donde se está aplicando Slow Start y Congestion Avoidance

- ☐ **a)** SS:[6,16],[17,22]; CA:[1-6],[23,26]
- ☐ **b)** SS:[4-6],[22,26]; CA:[6,16],[17,23]
- ☐ **c)** SS:[1-6],[23,26]; CA:[6,16],[17,22]
- ☐ **d)** SS:[1-16]; CA:[17,26]

> **8** (1p) ¿Qué ha pasado en la ronda 16?

- ☐ **a)** Se ha alcanzado el threshold.
- ☐ **b)** Ha expirado un timeout.
- ☐ **c)** Se han recibido 3 ACKs duplicados.
- ☐ **d)** El receptor ha reducido su ventana a 21 MSS.

> **9** (1p) ¿Qué ha pasado en la ronda 22?

- ☐ **a)** Se ha alcanzado el threshold.
- ☐ **b)** Ha expirado un timeout.
- ☐ **c)** Se han recibido 3 ACKs duplicados.
- ☐ **d)** El receptor ha reducido su ventana a 1 MSS.

> **10** (1p) ¿Cuál es el valor del threshold (ssthresh) inicial (medido en MSS)?

- ☐ **a)** 1
- ☐ **b)** 26
- ☐ **c)** 32
- ☐ **d)** 42

> **11** (1p) ¿Cuál es el valor de threshold en la ronda 18 de transmisión?

- ☐ **a)** La que indique el receptor.
- ☐ **b)** 32 MSS.
- ☐ **c)** 24 MSS.
- ☐ **d)** 42 MSS.

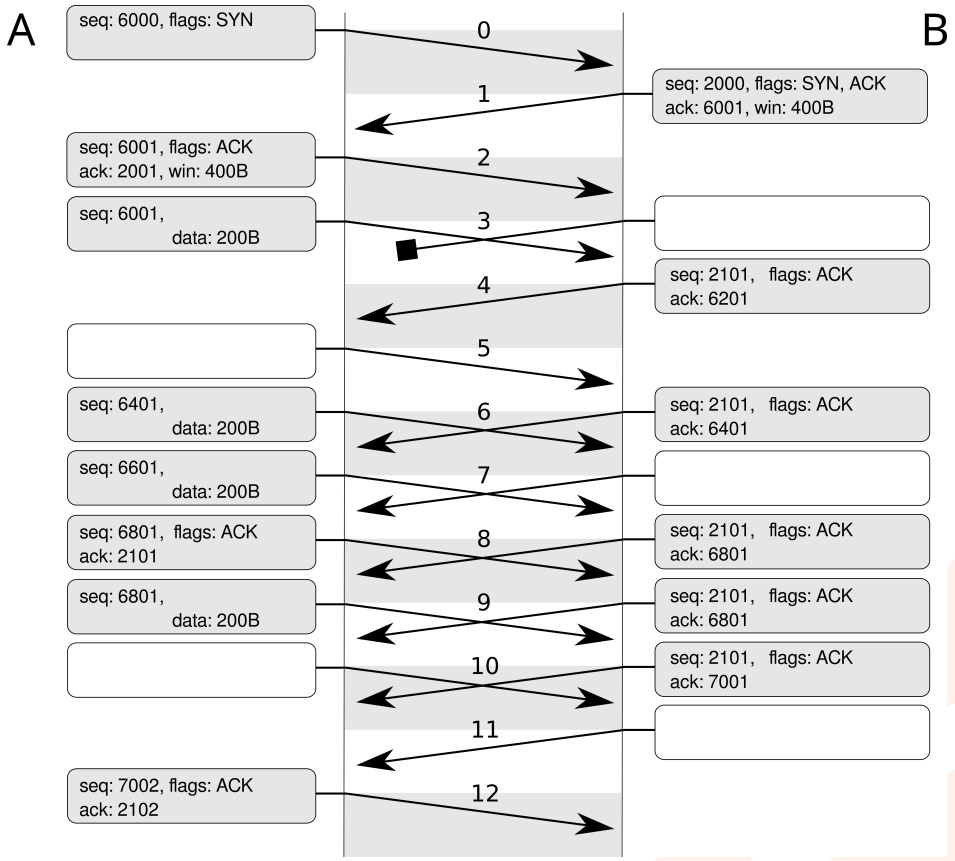
> **12** (1p) ¿Cuál es el valor de threshold en la ronda 24 de transmisión?

- ☐ **a)** La que indique el receptor.
- ☐ **b)** 13 MSS.
- ☐ **c)** 1 MSS.
- ☐ **d)** 26 MSS.

> **13** (1p) Asumiendo que rwnd>cwnd en todo la conexión ¿Cuáles serán los valores de cwnd y ssthresh si se detecta pérdida de paquetes después de la ronda 26 por recepción de 3 ACKs duplicados?

- ☐ **a)** cwnd=8, ssthresh=4
- ☐ **b)** cwnd=4, ssthresh=13
- ☐ **c)** cwnd=16, ssthresh=8
- ☐ **d)** cwnd=1, ssthresh=4

B [7p] En la figura aparece el inicio de una conexión TCP en la que se cumplen las siguientes condiciones: No se está aplicando el mecanismo de control de congestión. El plazo de retransmisión de A y B no cambia, pero puede ser diferente para cada uno. A y B pueden cambiar el valor de la ventana de recepción en cualquier momento. A y B pueden usar distintos tamaños de segmento siendo MSS=400B. Enviarán tanto como puedan y siempre que puedan, pero coincidiendo con un tick de reloj.



> **14** (1p) ¿Qué envió A en el instante 4?

- ☐ **a)** seq:1001, win:200B
- ☐ **b)** seq:1101, data:100B, flags:ACK, ack:8101.
- ☐ **c)** seq:1101, data:200B, flags:ACK, ack:8101.
- ☐ **d)** seq:1201, data:100B, flags:ACK, ack:8101.

> **15** (1p) ¿Qué envió B en el instante 5?

- ☐ **a)** seq:8001, data:100B, flags:ACK, ack:1301.
- ☐ **b)** seq:8101, data:200B, flags:ACK, ack:1301.
- ☐ **c)** seq:8201, data:100B, flags:ACK, ack:1301.
- ☐ **d)** Nada, el segmento de A en el tick 4 estaba corrupto.

> **16** (1p) ¿Qué envió A en el instante 7

- ☐ **a)** seq: 1001, data: 100B, flags:ACK, ack:8101.
- ☐ **b)** seq: 1101, data: 200B, flags:ACK, ack:8101.
- ☐ **c)** seq: 1201, data: 100B, flags:ACK, ack:8101.
- ☐ **d)** El segmento de B en el tick 5 estaba corrupto.

> **17** (1p) ¿Cuál es el plazo de retransmisión de A expresado en ticks de reloj?

- ☐ **a)** 1
- ☐ **b)** 2
- ☐ **c)** 3
- ☐ **d)** Indeterminado.

> **18** (1p) ¿Cuál es el plazo de retransmisión de B expresado en ticks de reloj?

- ☐ **a)** 1
- ☐ **b)** 2
- ☐ **c)** 3
- ☐ **d)** No hay ninguna RTX.

> **19** (1p) ¿Por qué B no envía datos en el segmento del tick 10?

- ☐ **a)** La ventana de envío de B está llena.
- ☐ **b)** La ventana de recepción de A está llena.
- ☐ **c)** La ventana de recepción de B está llena.
- ☐ **d)** B ha terminado de enviar datos.

> **20** (1p) ¿Por qué A solo envía 50B en el segmento del tick 11?

- ☐ **a)** La ventana de envío de B está llena.
- ☐ **b)** La ventana de recepción de A está llena.
- ☐ **c)** La ventana de recepción de B está llena.
- ☐ **d)** Es el resto que le queda por enviar.